

1] ARRAY :

A] Two pointers

I] opposite ends

You use two indices (pointers):

- One starts from the left end of the array.
- The other starts from the right end.

Then you move them toward each other based on some condition.

When should you think of Opposite-End Two Pointers?

1. Sorted Array
2. Pair-related questions e,g- pair sum, closest pair, count pair
3. Palindrome problems
4. Reverse an array/string

Template :

```
int L = 0;
int R = n - 1;

while (L < R) {

    if (condition satisfied) {
        // answer found
    }
    else if (need larger value) {
        L++;
    }
    else {
        R--;
    }
}
```

II] Same Direction (Fast & Slow)

The fast pointer explores the array.

The slow pointer builds or tracks the answer

Use this pattern when:

- You process elements **in order**
- You want to **compare or relate two positions**
- You need to **skip, filter, or track a range**
- You want to avoid nested loops ($O(N^2)$)

Typical Problem Statements

- Remove duplicates from sorted array
- Check if array is a subsequence of another
- Longest substring without repeating characters

- Merge two sorted arrays
- Partition array based on condition
- Fast & slow pointer problems (variant)

General Algorithm Template

initialize slow = 0

for fast in range(0, n):

if condition satisfied:

perform operation using slow & fast

slow += 1

Both pointers move forward, but:

- **fast** moves **every iteration**
- **slow** moves **only when needed**

Example 1: Remove Duplicates from Sorted Array

2: Check Subsequence

3: Longest Substring Without Repeating Characters

4: Merge Two Sorted Arrays

Ask yourself:

- Do I need to scan forward once?
- Can one pointer track progress?
- Is order important?
- Can I replace a nested loop? If yes → Two Pointers (Same Direction)

II] Partition/ dutch national flag

Concept

The Dutch National Flag problem divides the array into three partitions:

- **Low**: contains all 0s
- **Mid**: contains all 1s
- **High**: contains all 2s

Approach

We use three pointers: low, mid and high. low tracks the boundary where 0s should end. mid traverses through the array. high tracks the boundary where 2s should begin.

The algorithm processes the array in one pass:

- **If the current element is 0**: swap it with the element at low, then increment both low and mid
- **If the current element is 1**: leave it in place and just move the mid pointer.
- **If the current element is 2**: swap it with the element at high and decrement high

```
def dutch_national_flag(arr):
    low, mid = 0, 0
    high = len(arr) - 1

    while mid <= high:
        if arr[mid] == 0:
            arr[low], arr[mid] = arr[mid], arr[low]
            low += 1
            mid += 1

        elif arr[mid] == 1:
            mid += 1

        else:
            arr[mid], arr[high] = arr[high], arr[mid]
            high -= 1

    return arr
```

T.C -O(n) SC-O(1)

DNF is a specialized/generalized form of the two-pointer partition technique

Used recursively in 3way quick sort

B] Sliding window

used for problems involving **contiguous subarrays or substrings**.

This “window” slides one element at a time:

- We **add** the new element that enters the window (from the right)
- We **remove** the element that leaves the window (from the left)

This avoids recalculating the entire window every time.

Keywords:

- longest
- shortest/minimum
- maximum sum
- count subarrays
- substring
- consecutive elements

I] Fixed size

```
int l=0;
int window=0;

for(int r=0;r<n;r++){

    window += nums[r];

    if(r-l+1 > k){
        window -= nums[l];
        l++;
    }

    if(r-l+1 == k){
        // process answer
    }
}
```

II] Variable Size Sliding Window

A] Expand-Shrink Window

Approach:

1. Expand -> r++
2. Condition breaks?
3. Shrink -> l++
4. Condition valid again?
5. Continue expanding

When to use

Whenever you see

longest ,shortest ,minimum ,maximum ,at most K , exactly K , distinct characters ,sum ≤ K ,frequency constraints

```
int l=0;

for(int r=0;r<n;r++){

    // include nums[r]

    while(window_invalid){
        // remove nums[l]
        l++;
    }

    ans = max(ans , r-l+1);
}
```

B] Monotonic window

A Monotonic Window is a sliding window technique where we maintain the elements of the current window in sorted (monotonic) order using a deque.

When do we use it?

Whenever the problem asks something like

- Maximum in every window

- Minimum in every window
- First greater/smaller element inside a window
- Maintain sorted order while sliding

The key difference is:

Normal sliding window

- Just keeps frequency/count/sum.

Monotonic window

- Maintains elements in increasing/decreasing order using a deque.

A **deque (Double Ended Queue)** is a linear data structure that allows **insertion and deletion from both the front and the back** in **O(1)** time

Operations		
Operation	Description	Time Complexity
<code>push_front(x)</code>	Insert element at the front	O(1)
<code>push_back(x)</code>	Insert element at the back	O(1)
<code>pop_front()</code>	Remove front element	O(1)
<code>pop_back()</code>	Remove last element	O(1)
<code>front()</code>	Access first element	O(1)
<code>back()</code>	Access last element	O(1)
<code>size()</code>	Number of elements	O(1)
<code>empty()</code>	Check if deque is empty	O(1)

```
deque<int> dq;
```

- ☐ Deque = Double Ended Queue.
- ☐ Supports insertion and deletion from both ends in **O(1)**.
- ☐ In sliding window problems, **store indices, not values**.
- ☐ Remove expired indices from the **front**.
- ☐ Remove smaller (or larger) useless elements from the **back**.
- ☐ A monotonic deque ensures each element is inserted and removed at most once, giving an overall **O(n)** time complexity.

Complete Template (Maximum)

```
</> C++
deque<int> dq;

for (int i = 0; i < n; i++) {

    // Remove expired indices
    while (!dq.empty() && dq.front() <= i - k)
        dq.pop_front();

    // Remove smaller elements
    while (!dq.empty() && nums[dq.back()] <= nums[i])
        dq.pop_back();

    dq.push_back(i);

    if (i >= k - 1)
        ans.push_back(nums[dq.front()]);
}
```

For Minimum

```
nums[dq.back()] >= nums[i]
```

C] PREFIX BASED :

preprocessing cumulative information so every range query becomes **O(1)**.

I] Prefix sum

Store the sum from the beginning up to each index.

```
pref[i] = pref[i-1] + arr[i]
```

Range sum : L...R

```
sum = pref[R] - pref[L-1]
```

Common problems

- Range Sum Query
- Subarray Sum Equals K
- Continuous Subarray Sum
- Pivot Index
- Maximum Size Subarray Sum K

II]Prefix xor

Exactly the same idea.

Instead of addition,

use XOR.

```
pref[i] = pref[i-1] ^ arr[i]
```

Range XOR

```
xor(L,R)
= pref[R] ^ pref[L-1]
```

because

```
a ^ a = 0
```

Used in

- XOR Queries
- Maximum XOR tricks
- Bit manipulation problems

III] 2D Prefix Sum

For 2d matrices

Build

```
pref[i][j]
=
matrix[i][j]
+ top
+ left
- diagonal
```

Formula

```
pref[i][j]

=

a[i][j]
+ pref[i-1][j]
+ pref[i][j-1]
- pref[i-1][j-1]
```

D] SUBARRAY :

I] Maximum Subarray Sum (Kadane's Algorithm)

APPROACH:

Negative running sum?

Throw it away. Start fresh.

```
int maxSubArray(vector<int>& nums) {
    int curr = nums[0];
    int ans = nums[0];

    for(int i = 1; i < nums.size(); i++) {
        curr = max(nums[i], curr + nums[i]);
        ans = max(ans, curr);
    }

    return ans;
}
```

II] Maximum Product Subarray

```
int maxProduct(vector<int>& nums) {
    int mx = nums[0];
    int mn = nums[0];
    int ans = nums[0];

    for(int i = 1; i < nums.size(); i++) {

        int a = nums[i];
        int b = mx * nums[i];
        int c = mn * nums[i];

        mx = max({a,b,c});
        mn = min({a,b,c});

        ans = max(ans,mx);
    }

    return ans;
}
```

III] Subarray with Given XOR

E] BINARY SEARCH

I] Binary Search on Index (Search Space = Array Indices)

Here you're searching **where** something is in a sorted array.

Search space: 0 ... n-1

```
int l = 0, r = n - 1;

while (l <= r) {
    int mid = l + (r - l) / 2;

    if (arr[mid] == target)
        return mid;

    else if (arr[mid] < target)
        l = mid + 1;

    else
        r = mid - 1;
}

return -1;
```

First occurrence and last occurrence

First Occurrence

```
// C++

if (nums[mid] == target) {
    ans = mid;
    r = mid - 1;    // Search Left
}
```

Think:

"I found one, but maybe there's an earlier one."

Last Occurrence

```
// C++

if (nums[mid] == target) {
    ans = mid;
    l = mid + 1;    // Search right
}
```

Think:

"I found one, but maybe there's a later one."

Meaning	Condition	If Found
Lower Bound	First <code>>= target</code>	<code>ans = mid; r = mid - 1</code>
Upper Bound	First <code>> target</code>	<code>ans = mid; r = mid - 1</code>
First Occurrence	First <code>== target</code>	<code>ans = mid; r = mid - 1</code>
Last Occurrence	Last <code>== target</code>	<code>ans = mid; l = mid + 1</code>

II] Binary Search on Answer

When do we use it?

When the problem asks for:

- Minimum possible value
- Maximum possible value
- Smallest/Largest X satisfying some condition

Generic Template (Minimize Answer)

```
<> C++

int l = minimum_possible_answer;
int r = maximum_possible_answer;
int ans = -1;

while (l <= r) {
    int mid = l + (r - l) / 2;

    if (check(mid)) {
        ans = mid;
        r = mid - 1;    // Try a smaller answer
    }
    else {
        l = mid + 1;
    }
}

return ans;
```

Generic Template (Maximize Answer)

```
<> C++

int l = minimum_possible_answer;
int r = maximum_possible_answer;
int ans = -1;

while (l <= r) {
    int mid = l + (r - l) / 2;

    if (check(mid)) {
        ans = mid;
        l = mid + 1;    // Try a larger answer
    }
    else {
        r = mid - 1;
    }
}

return ans;
```

2]STRING

III] PATTERN MATCHING

A} KMP (failure function) Knuth-Morris-Pratt

Purpose: Find a pattern in a string in $O(n + m)$ time.

Idea

Instead of restarting from the beginning after a mismatch, KMP uses information about the pattern itself.

It builds an **LPS (Longest Prefix Suffix)** array (also called the **failure function**).

$LPS[i]$ = length of the longest proper prefix which is also a suffix for pattern[0...i]

Questions

- Find first occurrence
- Count occurrences
- Repeated substring pattern
- Shortest palindrome
- Longest happy prefix

LPS

```
1. len = 0, i = 1
2. If pattern[i] == pattern[len]:
    len++
    lps[i] = len
    i++
3. Else if len != 0:
    len = lps[len-1]
4. Else:
    lps[i] = 0
    i++
```

```
vector<int> buildLPS(string pat) {
    int m = pat.size();
    vector<int> lps(m);

    int len = 0;
    int i = 1;

    while (i < m) {

        if (pat[i] == pat[len]) {
            len++;
            lps[i] = len;
            i++;
        }
        else {

            if (len != 0) {
                len = lps[len - 1];
            }
            else {
                lps[i] = 0;
                i++;
            }
        }
    }

    return lps;
}
```

COMPLETE KMP

1. Build the LPS array for the pattern.
2. Start with $i = 0$ (text), $j = 0$ (pattern).
3. If characters match, increment both i and j .
4. If $j ==$ pattern length, record the match and set $j = lps[j-1]$.
5. On mismatch:
 - if $j != 0$, set $j = lps[j-1]$
 - else increment i .

```
vector<int> KMP(string text, string pat) {
    vector<int> lps = buildLPS(pat);
    vector<int> ans;

    int i = 0;
    int j = 0;

    while (i < text.size()) {

        if (text[i] == pat[j]) {
            i++;
            j++;
        }

        if (j == pat.size()) {
            ans.push_back(i - j);
            j = lps[j - 1];
        }
        else if (i < text.size() && text[i] != pat[j]) {

            if (j != 0)
                j = lps[j - 1];
            else
                i++;
        }
    }

    return ans;
}
```

Use KMP when:

- You need to find all occurrences of a pattern in a text.
- The text is very large and repeated comparisons would be expensive.
- You're solving competitive programming or interview problems involving efficient string matching

B) Rabin carp

string pattern matching algorithm that uses a rolling hash to find occurrences of a pattern in a text.

it compares the hash value of the pattern with the hash value of every substring of the same length.

If the hashes match, it then checks the actual characters to avoid false positives due to hash collisions.

Hash:

A hash of a string is a single integer computed from all the characters in the string, designed so that equal strings have equal hashes and different strings are very likely to have different hashes. It acts like a compact "fingerprint" of the string.

1] rolling hash:

A rolling hash is a hash function that lets you compute the hash of the next substring from the current substring's hash in $O(1)$ time, instead of recomputing it from scratch.

newHash =

$((oldHash - firstChar * power) * BASE + newChar)$

Feature	Rabin-Karp	KMP
Main idea	Rolling hash	LPS (prefix function)
Average time	$O(n + m)$	$O(n + m)$
Worst-case time	$O(nm)$ (hash collisions)	$O(n + m)$
Extra preprocessing	Pattern hash	LPS array
Best use case	Multiple pattern matching, plagiarism detection, substring search	Single-pattern matching with guaranteed linear time

1. Compute pattern hash.
2. Compute first window hash.
3. Compare hashes.
4. If hashes match, compare characters.
5. Roll the window hash.

```
newHash = (oldHash - outgoing * power) % MOD;
newHash = (newHash + MOD) % MOD; // avoid negative
newHash = (newHash * BASE + incoming) % MOD;
```

```
class Solution {
public:
    vector<int> rabinKarp(string text, string pattern) {
        const long long BASE = 31;
        const long long MOD = 1000000007;

        int n = text.size();
        int m = pattern.size();

        vector<int> ans;
        if (m > n) return ans;

        long long patternHash = 0;
        long long windowHash = 0;
        long long power = 1;

        // BASE^(m-1)
        for (int i = 0; i < m - 1; i++)
            power = (power * BASE) % MOD;

        // Initial hashes
        for (int i = 0; i < m; i++) {
            patternHash = (patternHash * BASE + (pattern[i] - 'a' + 1)) % MOD;
            windowHash = (windowHash * BASE + (text[i] - 'a' + 1)) % MOD;
        }

        for (int i = 0; i <= n - m; i++) {

            if (patternHash == windowHash) {
                bool match = true;

                for (int j = 0; j < m; j++) {
                    if (text[i + j] != pattern[j]) {
                        match = false;
                        break;
                    }
                }

                if (match)
                    ans.push_back(i);
            }

            if (i < n - m) {
                windowHash = (windowHash -
                    (text[i] - 'a' + 1) * power % MOD + MOD) % MOD;

                windowHash = (windowHash * BASE +
                    (text[i + m] - 'a' + 1)) % MOD;
            }
        }

        return ans;
    }
};
```

C) Z Algorithm

What is the Z-array?

For every index i , $Z[i]$ tells us: Length of the longest substring starting at i that matches the prefix of the string

The Complete Algorithm

Maintain

L
R

For every i :

```
if(i>R)
    compare normally
else
    k=i-L
    if(Z[k]<R-i+1)
        Z[i]=Z[k]
    else
        L=i
        compare from R+1
        update R
```

```
vector<int> zFunction(string s) {
    int n = s.size();
    vector<int> z(n);

    int L = 0, R = 0;

    for (int i = 1; i < n; i++) {

        if (i <= R)
            z[i] = min(R - i + 1, z[i - L]);

        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            z[i]++;

        if (i + z[i] - 1 > R) {
            L = i;
            R = i + z[i] - 1;
        }
    }

    return z;
}
```

Using Z Algorithm for Pattern Matching

Instead of implementing a separate search, concatenate:

pattern + "\$" + text

For example:

```
Pattern = "abc"
Text    = "abxabcabc"

Combined:

abc$abxabcabc
```

Compute the Z-array. Whenever:

$Z[i] == \text{pattern.length}()$

you've found an occurrence of the pattern in the text.

```
vector<int> zFunction(string s) {
    int n = s.size();
    vector<int> z(n);

    int L = 0, R = 0;

    for (int i = 1; i < n; i++) {

        if (i <= R)
            z[i] = min(R - i + 1, z[i - L]);

        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            z[i]++;

        if (i + z[i] - 1 > R) {
            L = i;
            R = i + z[i] - 1;
        }
    }

    return z;
}

vector<int> search(string text, string pattern) {

    string s = pattern + "$" + text;

    vector<int> z = zFunction(s);

    vector<int> ans;

    int m = pattern.size();

    for (int i = 0; i < z.size(); i++) {
        if (z[i] == m)
            ans.push_back(i - m - 1);
    }

    return ans;
}
```



3] Hashing

A] Frequency based

Idea: Store how many times each element appears.

```
</> C++  
  
unordered_map<int, int> freq;  
  
for (int x : nums)  
    freq[x]++;
```

Used for

- Counting occurrences
- Finding duplicates
- Majority element
- Anagrams
- Top K frequent

B] Lookup based

Idea: While iterating, check if something needed already exists.

```
</> C++  
  
unordered_map<int,int> mp;  
  
for(int i=0;i<n;i++){  
    if(mp.count(target-nums[i]))  
        return true;  
  
    mp[nums[i]]=i;  
}
```

Used for

- Complement search
- Fast existence checking

C] Set based

Uses

```
</> C++  
  
unordered_set<int>
```

instead of map.

When only existence matters.

```
</> C++  
  
unordered_set<int> st;  
  
for(int x:nums)  
    st.insert(x);
```

Used for

- O(1) existence
- Removing duplicates

4. Index Mapping

Store where something first/last appeared.

```
</> C++  
  
unordered_map<int,int> firstIndex;
```

or

```
</> C++  
  
unordered_map<char,int> lastSeen;
```

Used for

- Prefix sums
- Sliding window
- First occurrence
- Last occurrence

E] Grouping pattern

Group multiple elements having the same property.

```
</> C++  
  
unordered_map<string, vector<string>> groups;
```

Example

```
eat  
tea  
ate  
  
↓  
  
"aet" -> [eat, tea, ate]
```

Used for

- Categorization
- Bucketing

4] STACK

linear data structure that follows the LIFO principle

The last element you put into the stack is the first one that comes out.

```
#include <stack>  
using namespace std;  
  
stack<int> st;  
  
st.push(10);  
st.push(20);  
  
cout << st.top() << endl; // 20  
  
st.pop();  
  
cout << st.top() << endl; // 10  
  
cout << st.size() << endl; // 1  
  
cout << st.empty(); // 0 (false)
```

A] What is a Monotonic Stack?

A monotonic stack is simply a normal stack with one extra rule:

The elements in the stack must always remain in increasing or decreasing order.

Whenever a new element would break that order, we remove (pop) elements until the order is restored.

1) INCREASING :

Rule

Before pushing `x` :

```
</> C++  
  
while(!st.empty() && st.top() >= x)  
    st.pop();  
  
st.push(x);
```

2) DECREASING

Rule

```
</> C++  
  
while(!st.empty() && st.top() <= x)  
    st.pop();  
  
st.push(x);
```

Q1. Am I looking for Greater or Smaller?

- Greater → Decreasing stack → pop `<=`
- Smaller → Increasing stack → pop `>=`

Q2. Am I looking for Next or Previous?

- Next → Traverse Right → Left
- Previous → Traverse Left → Right

B] RANGE/SPAN ELEMENT:

```
Find Previous Boundary  
+  
Find Next Boundary  
↓  
Compute Width / Span  
↓  
Apply Problem Formula
```

Problem	Left Boundary	Right Boundary
Histogram	Previous Smaller	Next Smaller
Stock Span	Previous Greater	—
Subarray Minimums	Previous Smaller	Next Smaller
Subarray Maximums	Previous Greater	Next Greater

Need Previous?

→ Left → Right

Need Next?

→ Right → Left

Histogram

Previous Smaller

Next Smaller

width = right-left-1

area = height×width

Stock Span

Previous Greater

span = i-prevGreater

Subarray Minimums

Previous Smaller

Next Smaller

left = i-prev

right = next-i

answer += value×left×right

Subarray Maximums

Previous Greater

Next Greater

left = i-prev

right = next-i

answer += value×left×right

5] TREES

A Tree is a data structure made of nodes connected by edge

Each circle is a Node.

Each line is an Edge.

Edges = Nodes – 1

A] Binary tree

A Binary Tree is a tree where every node has at most 2 children

3. TreeNode in C++

Every node stores

- value
- left child
- right child

```
<> C++  
  
struct TreeNode  
{  
    int val;  
    TreeNode* left;  
    TreeNode* right;  
  
    TreeNode(int x)  
    {  
        val=x;  
        left=NULL;  
        right=NULL;  
    }  
};
```

What does `TreeNode* left` mean?

Suppose

```
5  
/  
2
```

Memory

Address	Value
100	----- Node(5)
200	----- Node(2)

Node 5

```
val = 5  
  
left = 200  
  
right = NULL
```

Node 2

```
val = 2  
  
left = NULL  
  
right = NULL
```

Each child is another **TreeNode object** stored somewhere else in memory

It stores the **address** of node , not the value

So

`TreeNode* left;`

`TreeNode* right;`

stores address not value so it's a pointer

4. Creating Nodes

```
<> C++  
  
TreeNode* root = new TreeNode(10);  
  
TreeNode* a = new TreeNode(5);  
  
TreeNode* b = new TreeNode(20);
```

Connect them

```
<> C++  
  
root->left = a;  
root->right = b;
```

Tree becomes

```
  10  
 /  \  
5    20
```

Leaf Node-Node with no children.

Internal Node - Any node having at least one child.

Degree-Number of children.

Level- Distance from root in terms of edges, starting at 0.

Depth of a node = number of edges from root to that node.

Depth and level are often used interchangeably in interview questions

Height = longest path downward from a node to any leaf, measured in edges.

Easy Trick

Depth

↑ Towards Root

Height

↓ Towards Leaf

Height of Tree-Height of root.

Subtree

Take any node., Include all its descendants., That forms a subtree.

In a **tree**, you move using pointers (node->left, node->right).



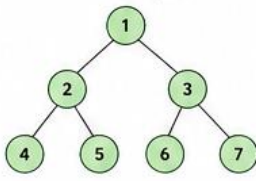
TREES – VISUAL SUMMARY (ALL CONCEPTS)



1. WHAT IS A TREE?

- A tree is a connected data structure with no cycles.
- Exactly one path between any two nodes.
- For N nodes, number of edges = $N - 1$

Example:



Nodes = 7 Edges = 6 (Edges = Nodes - 1)

2. BINARY TREE

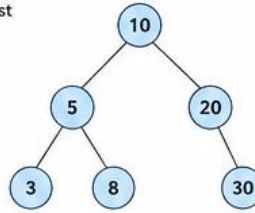
A tree in which each node has at most 2 children.

Allowed:

- ✓ 0 child
- ✓ 1 child
- ✓ 2 children

Not allowed:

- ✗ 3 children or more

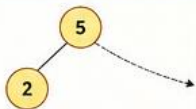


3. TREENODE IN C++

```
struct TreeNode{
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x){
        val = x;
        left = NULL;
        right = NULL;
    }
};
```

left and right store ADDRESS (pointer to child), not the value.

4. MEMORY REPRESENTATION



Memory (Example)

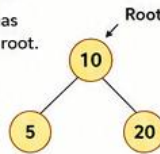
Address	Value	Left	Right
100	5	200	NULL
200	2	NULL	NULL

Node 5: left = 200 (address of node 2)
Node 2: left = NULL, right = NULL

5. ROOT

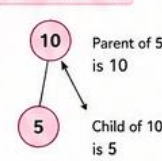
Topmost node.

Every tree has exactly one root.

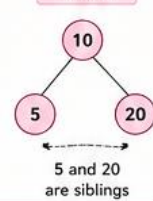


6. PARENT, CHILD, SIBLINGS

Parent & Child

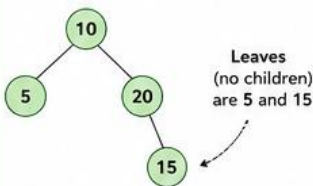


Siblings



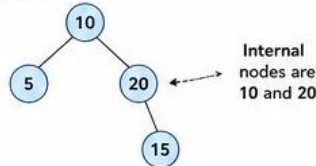
7. LEAF NODE

A node with no children.



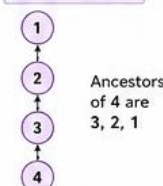
8. INTERNAL NODE

A node with at least one child.

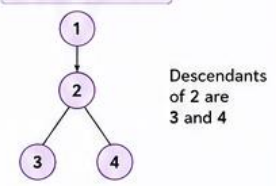


9. ANCESTOR & DESCENDANT

Ancestors of 4

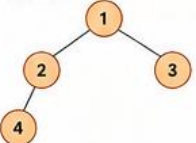


Descendants of 2



10. DEGREE

Number of children a node has.

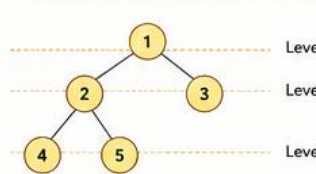


Degree

deg(1) = 2
deg(2) = 1
deg(3) = 0
deg(4) = 0

Max degree in binary tree = 2

11. LEVEL (Starting from 0)



Node	Level
1	0
2	1
3	1
4	2
5	2

Level = distance from root in terms of edges (starting at 0)

12. DEPTH (of a node)



Node	Depth
1	0
2	1
3	2

Depth = number of edges from root to the node

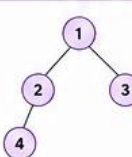
13. HEIGHT (of a node)



Node	Height
1	2
2	1
3	0

Height = longest path downward from the node to a leaf (in edges)

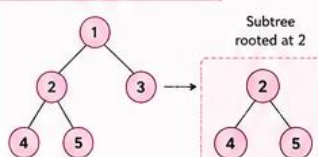
14. HEIGHT OF TREE



Longest path from root to a leaf:
1 → 2 → 4 (2 edges)

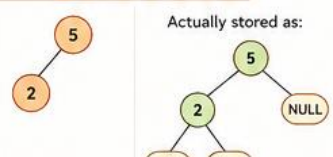
Height of the tree = 2

15. SUBTREE



A subtree includes a node and all its descendants.

16. NULL CHILD



Actually stored as:

Missing children are represented by NULL.

17. SUMMARY TABLE

Term	Meaning
Root	Topmost node. Only one in a tree.
Parent	Node directly above another node.
Child	Node directly below another node.
Sibling	Nodes having the same parent.
Leaf	Node with no children.
Internal Node	Node having at least one child.
Ancestor	Any node on the path from root to the node (excluding itself).
Descendant	Any node in the subtree of that node (excluding itself).
Degree	Number of children a node has.
Level	Distance from root (in edges), starting from 0.
Depth	Number of edges from root to the node.
Height	Longest path from the node down to a leaf (in edges).
Subtree	A node and all of its descendants.

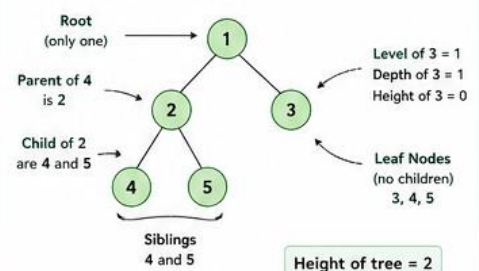
18. HOW RECURSION WORKS ON TREES

Every tree problem (DFS) uses this idea.

```
void solve(TreeNode* node)
{
    if(node == NULL) return;
    // do something with node
    solve(node->left);
    solve(node->right);
}
```

Base Case: node == NULL
Stop when we reach a NULL child.
This is the key to almost all tree algorithms.

19. QUICK RECAP DIAGRAM



KEY TAKEAWAYS

- ✓ Trees have no cycles and $N - 1$ edges.
- ✓ Nodes are connected using pointers.

- ✓ Understand depth vs height.
- ✓ NULL represents missing children.

- ✓ Recursion is the heart of tree problems.
- ✓ Practice traversals next!



B] DFS Traversals

DFS = Depth First Search

Instead of visiting nodes level by level, we go as deep as possible into one branch before backtracking.

DFS first goes down the left branch before exploring the right.

The Three DFS Traversals

Every DFS traversal differs only in **when you process the current node**.

There are exactly **three places** where you can visit a node:

- node
- (1) Before going left
- node
- (2) Between left & right
- node
- (3) After going right

That's why there are exactly **3 DFS traversals**.

1. Preorder

Order:

Root
Left
Right

Short form:

N L R

Recursive code

```
<> C++  
void preorder(TreeNode* root)  
{  
    if(root==NULL)  
        return;  
  
    cout<<root->val<<" ";  
  
    preorder(root->left);  
  
    preorder(root->right);  
}
```

2. Inorder

Order

Left
Root
Right

Short

L N R

```
void inorder(TreeNode* root)  
{  
    if(root==NULL)  
        return;  
  
    inorder(root->left);  
  
    cout<<root->val<<" ";  
  
    inorder(root->right);  
}
```

3. Postorder

Order

Left
Right
Root

Short

L R N

```
void postorder(TreeNode* root)  
{  
    if(root==NULL)  
        return;  
  
    postorder(root->left);  
  
    postorder(root->right);  
  
    cout<<root->val<<" ";  
}
```

Why Recursion Works So Naturally

Consider this tree:

```
  1  
 / \  
2   3
```

Call:

```
<> C++  
preorder(root);
```

The execution unfolds like this:

```
preorder(1)  
|  
├─ print 1  
├─ preorder(2)  
|   ├─ print 2  
|   └─ preorder(NULL)  
|   └─ preorder(NULL)  
└─ preorder(3)  
    ├─ print 3  
    └─ preorder(NULL)  
    └─ preorder(NULL)
```

Notice that every subtree is itself a tree, so the same function solves the left subtree and the right subtree.

This is the heart of tree recursion.

When Are They Used?

Preorder

- Copy a tree
- Serialize a tree
- Prefix expressions

Inorder

- In a **Binary Search Tree**, inorder gives values in **sorted order**.
- Very common in BST problems.

Postorder

- Delete/free a tree
- Calculate height
- Compute subtree answers before using the parent

Universal DFS Template

Every recursive tree problem starts like this:

```
</> C++  
  
void dfs(TreeNode* root)  
{  
    if(root==NULL)  
        return;  
  
    // Before left  
    dfs(root->left);  
  
    // Between left & right  
    dfs(root->right);  
  
    // After right  
}
```

The only difference between preorder, inorder, and postorder is where you place the processing statement (cout, computation, update, etc.).

Recursion in DFS uses a stack internally.

The iterative versions simply use that stack explicitly

Trees — Lesson 3: Iterative DFS Traversals

Preorder (Iterative)

```
Push root  
  
while(stack not empty)  
  
    pop top  
  
    print it  
  
    push right  
  
    push left
```

Right before left because stack is LIFO

```
vector<int> preorderTraversal(TreeNode* root)  
{  
    vector<int> ans;  
  
    if(root==NULL)  
        return ans;  
  
    stack<TreeNode*> st;  
  
    st.push(root);  
  
    while(!st.empty())  
    {  
        TreeNode* node=st.top();  
        st.pop();  
  
        ans.push_back(node->val);  
  
        if(node->right)  
            st.push(node->right);  
  
        if(node->left)  
            st.push(node->left);  
    }  
  
    return ans;  
}
```

Inorder (Iterative)

Key Idea

Keep going left until impossible.

Then process.

Then go right.

Algo:

```
current=root  
  
while(current || stack not empty)  
  
    while(current)  
  
        push(current)  
  
        current=current->left  
  
    current=stack.top()  
  
    pop()  
  
    print  
  
    current=current->right
```

```
vector<int> inorderTraversal(TreeNode* root)
{
    vector<int> ans;

    stack<TreeNode*> st;

    TreeNode* curr=root;

    while(curr || !st.empty())
    {
        while(curr)
        {
            st.push(curr);
            curr=curr->left;
        }

        curr=st.top();
        st.pop();

        ans.push_back(curr->val);

        curr=curr->right;
    }

    return ans;
}
```

Postorder (Iterative)

This is the hardest traversal iteratively.

There are two common approaches:

Method 1 (Easy)

Use 2 stacks.

Algorithm

```
Push root into stack1

while(stack1)

    pop

    push into stack2

    push left

    push right

Finally

Pop everything from stack2
```

```
vector<int> postorderTraversal(TreeNode* root)
{
    vector<int> ans;

    if(root==NULL)
        return ans;

    stack<TreeNode*> st1;
    stack<TreeNode*> st2;

    st1.push(root);

    while(!st1.empty())
    {
        TreeNode* node=st1.top();
        st1.pop();

        st2.push(node);

        if(node->left)
            st1.push(node->left);

        if(node->right)
            st1.push(node->right);
    }

    while(!st2.empty())
    {
        ans.push_back(st2.top()->val);
        st2.pop();
    }

    return ans;
}
```

Method 2

Use one stack.

This is an advanced interview topic and much trickier.

We'll learn it later after more recursion practice.

C] Breadth First Search (BFS) / Level Order Traversal

Instead of depth, BFS visits level by level.

Why do we need a Queue?

Algorithm

```
Push root into queue

while queue not empty

    Take front node

    Visit it

    Push left child

    Push right child
```

```
vector<int> levelOrder(TreeNode* root)
{
    vector<int> ans;

    if(root == NULL)
        return ans;

    queue<TreeNode*> q;

    q.push(root);

    while(!q.empty())
    {
        TreeNode* node = q.front();
        q.pop();

        ans.push_back(node->val);

        if(node->left)
            q.push(node->left);

        if(node->right)
            q.push(node->right);
    }

    return ans;
}
```

DFS vs BFS

DFS	BFS
Stack / Recursion	Queue
Goes deep	Goes level by level
Left subtree first	Entire level first
Pre/In/Postorder	Level Order

D] Recursion pattern

1. Top-Down Recursion (Preorder Style)

The parent computes or updates something before going to its children.

Example:

```
<> C++

void dfs(TreeNode* root, int depth)
{
    if(root == NULL)
        return;

    ans = max(ans, depth);

    dfs(root->left, depth + 1);
    dfs(root->right, depth + 1);
}
```

2. Bottom-Up Recursion (Postorder Style)

he parent waits for both children to finish.

Then it combines their answers.

Example:

```
int maxDepth(TreeNode* root)
{
    if(root == NULL)
        return 0;

    int left = maxDepth(root->left);
    int right = maxDepth(root->right);

    return 1 + max(left, right);
}
```

☐ Top-Down: ~30% of tree problems

☐ Bottom-Up: ~70% of tree problems.

Binary Search Tree (BST)

Definition

A Binary Search Tree is a binary tree where, for every node:

- All nodes in the left subtree have values less than the current node.
- All nodes in the right subtree have values greater than the current node.
- Both left and right subtrees are also BSTs.

Inorder Traversal of BST: Always gives sorted order.

Searching in BST

```
TreeNode* searchBST(TreeNode* root, int val) {
    if(root == NULL || root->val == val)
        return root;

    if(val < root->val)
        return searchBST(root->left, val);

    return searchBST(root->right, val);
}
```

Balanced : $O(\log n)$

Worst case (skewed): $O(n)$

Insert in BST

```
TreeNode* insertIntoBST(TreeNode* root, int val) {
    if(root == NULL)
        return new TreeNode(val);

    if(val < root->val)
        root->left = insertIntoBST(root->left, val);
    else
        root->right = insertIntoBST(root->right, val);

    return root;
}
```

Delete in BST

this is the most important BST operation.

Case 1: Leaf Node

```
  8
 /
3
```

Delete 3

```
  8
```

Simply remove it.

Case 2: One Child

```
  8
 /
3
 /
1
```

Delete 3

```
  8
 /
1
```

Replace the node with its child.

Case 3: Two Children

```
  8
 / \
3   10
 / \
1   6
```

Delete 3.

You cannot directly remove it because you'd lose both subtrees.

Instead:

1. Find the inorder successor (smallest in the right subtree) or the inorder predecessor (largest in the left subtree).
2. Copy its value into the node.
3. Delete that successor/predecessor node.

```
TreeNode* deleteNode(TreeNode* root, int key) {

    if (root == NULL)
        return NULL;

    if (key < root->val) {
        root->left = deleteNode(root->left, key);
    }
    else if (key > root->val) {
        root->right = deleteNode(root->right, key);
    }
    else {
        // Case 1: No left child
        if (root->left == NULL) {
            TreeNode* temp = root->right;
            delete root;
            return temp;
        }

        // Case 2: No right child
        if (root->right == NULL) {
            TreeNode* temp = root->left;
            delete root;
            return temp;
        }

        // Case 3: Two children
        TreeNode* curr = root->right;
        while (curr->left != NULL) {
            curr = curr->left;
        }

        root->val = curr->val;
        root->right = deleteNode(root->right, curr->val);
    }

    return root;
}
```

Generic Range Template

```
<> C++

bool solve(TreeNode* root, long long low, long long high) {

    // Base Case
    if (root == NULL)
        return true;

    // Node should lie within the valid range
    if (root->val <= low || root->val >= high)
        return false;

    // Left subtree: upper bound becomes current node
    // Right subtree: Lower bound becomes current node
    return solve(root->left, low, root->val) &&
        solve(root->right, root->val, high);
}
```

Call it with:

```
<> C++

return solve(root, LLONG_MIN, LLONG_MAX);
```

6] Recursion :

Recursion is a function calling itself until a base case stops further calls

Every recursive function has two parts:

1. Base Case → stopping condition
2. Recursive Call → smaller version of the same problem

Pattern	Problems
Single recursion	Print, Sum, Factorial
Functional recursion	Sum, Factorial
Multiple recursion	Fibonacci
Take / Don't Take	Subsequences
Same index	Combination Sum
For-loop recursion	Subsets II, Combination Sum II
Visited array	Permutations
Grid DFS	Word Search
Substring recursion	Palindrome Partition
State Tracking + Pruning	N-Queens, Sudoku

1. Basic Recursion

Multiple Recursive Calls:

A function calling itself until a base case stops further recursion

```
void solve(int n){  
  
    if(base_case)  
        return;  
  
    // Current Work  
  
    solve(smaller_problem);  
}
```

2. Parameterized Recursion

Carry answer in parameters.

```
void sum(int i,int n,int s){  
  
    if(i>n){  
        cout<<s;  
        return;  
    }  
  
    sum(i+1,n,s+i);  
}
```

3. Functional Recursion

```
int sum(int n){  
  
    if(n==0)  
        return 0;  
  
    return n+sum(n-1);  
}
```

4. Two Pointer Recursion

```
if(i>=j)  
    return;  
  
swap(a[i],a[j]);  
  
solve(i+1,j-1);
```

5. Multiple Recursion

```
return fib(n-1)+fib(n-2);
```

Tree

```
      f(5)  
     /  \  
    f(4) f(3)
```

Backtracking

Golden Rule

Choose

↓

Explore

↓

Undo

Every problem follows

```
“> C++
```

Do Choice

Recursive Call

Undo Choice

```

void solve(...){

    // Base case
    if(...){
        // store answer
        return;
    }

    // Choose
    ...

    // Explore
    solve(...);

    // Undo (Backtrack)
    ...

}

```

6. Take / Don't Take Pattern

Take

↓

Recursive

↓

Undo

↓

Don't Take

```

ds.push_back(arr[index]);

solve(index+1);

ds.pop_back();

solve(index+1);

```

11. For Loop Backtracking

Used when

Choices begin from

```
<> C++
```

index

Template

```
<> C++
```

```

for(int i=index;i<n;i++){

    Choose

    solve(i+1);

    Undo

}

```

12. Duplicate Skipping

Always

```
<> C++
```

```
sort(nums.begin(),nums.end());
```

Magic line

```
<> C++
```

```

if(i>index && nums[i]==nums[i-1])
    continue;

```

13. Combination Sum

Unlimited use.

Take

```
<> C++
```

```
solve(index,target-arr[index]);
```

Stay at same index.

Don't Take

```
<> C++
```

```
solve(index+1,target);
```

Rule

Can reuse

↓

Stay

Backtracking on a 2D Grid

```

if(invalid)
    return false;

```

Save state

Mark visited

Explore 4 directions

Restore state

Return result

```

class Solution {
public:

    bool dfs(vector<vector<char>>& board,
            string& word,
            int r,
            int c,
            int idx){

        if(idx == word.size())
            return true;

        if(r<0 || c<0 || r>=board.size() || c>=board[0].size())
            return false;

        if(board[r][c] != word[idx])
            return false;

        char temp = board[r][c];
        board[r][c] = '#';

        bool found =
            dfs(board, word, r+1, c, idx+1) ||
            dfs(board, word, r-1, c, idx+1) ||
            dfs(board, word, r, c+1, idx+1) ||
            dfs(board, word, r, c-1, idx+1);

        board[r][c] = temp;

        return found;
    }

    bool exist(vector<vector<char>>& board, string word) {

        int m = board.size();
        int n = board[0].size();

        for(int i=0; i<m; i++){

            for(int j=0; j<n; j++){

                if(dfs(board, word, i, j, 0))
                    return true;
            }
        }
    }
}

```

Step tracking and pruning

State = information about what has already happened.

What is Pruning?

Pruning means:

Stop exploring a branch as soon as you know it cannot lead to a valid solution.

Divide and Conquer

Divide a problem into smaller independent subproblems, solve them recursively, and combine the results.

3 steps:

1. Divide
2. Conquer
3. Combine

```

solve(problem){
    if(base case) return;

    divide();

    solve(left);
    solve(right);

    combine();
}

```

When do we use Divide and Conquer?

Look for statements like:

- Split array into halves
- Left part and right part
- Midpoint
- Merge results
- Independent subproblems

Merge sort

Instead of sorting the whole array at once, We keep dividing it into two halves until every part has only one element. Then we merge those sorted halves

Merge Algorithm

```

while(i<mid && j<=r)

    if(left<=right)

        push left

    else

        push right

while(left remains)

while(right remains)

```

That's the whole merge process.

```

class Solution {
public:

    void merge(vector<int>& arr, int l, int mid, int r)

        vector<int> temp;

        int i = l;
        int j = mid + 1;

        while (i <= mid && j <= r) {

            if (arr[i] <= arr[j]) {
                temp.push_back(arr[i]);
                i++;
            } else {
                temp.push_back(arr[j]);
                j++;
            }
        }

        while (i <= mid) {
            temp.push_back(arr[i]);
            i++;
        }

        while (j <= r) {
            temp.push_back(arr[j]);
            j++;
        }

        for (int k = l; k <= r; k++) {
            arr[k] = temp[k - l];
        }
    }

    void mergeSort(vector<int>& arr, int l, int r) {

        if (l >= r)
            return;

        int mid = l + (r - l) / 2;

        mergeSort(arr, l, mid);

        mergeSort(arr, mid + 1, r);

        merge(arr, l, mid, r);
    }

    vector<int> sortArray(vector<int>& nums) {

        mergeSort(nums, 0, nums.size() - 1);

        return nums;
    }
};

```

LINKEDLIST :

A Linked List is a linear data structure where elements are **not stored in contiguous memory**.

instead, every element (called a **Node**) stores:

- Data
- Address of the next node

Node Structure

```

struct ListNode {
    int val;
    ListNode *next;

    ListNode() {
        val = 0;
        next = NULL;
    }

    ListNode(int x) {
        val = x;
        next = NULL;
    }

    ListNode(int x, ListNode *next) {
        val = x;
        this->next = next;
    }
};

```

Array vs Linked List

Array	Linked List
Contiguous memory	Random memory
Fixed size	Dynamic size
O(1) indexing	O(n) indexing
Insertion costly	Easy insertion
Deletion costly	Easy deletion

5. Creating Nodes

Single node

```
</> C++
```

```
ListNode* head = new ListNode(10);
```

Two nodes

```
</> C++
```

```
ListNode* first = new ListNode(10);
```

```
ListNode* second = new ListNode(20);
```

```
first->next = second;
```

Three nodes

```
</> C++
```

```
ListNode* third = new ListNode(30);
```

```
second->next = third;
```

Now

```
10 → 20 → 30
```

6. Traversal

Pattern:

```
</> C++

ListNode* temp = head;

while(temp != NULL){
    cout << temp->val << " ";
    temp = temp->next;
}
```

7. Count Nodes

Exactly same traversal

```
</> C++

int count = 0;

ListNode* temp = head;

while(temp){
    count++;
    temp = temp->next;
}

return count;
```

Insert at Beginning

```
ListNode* insertFront(ListNode* head,int val){

    ListNode* node=new ListNode(val);

    node->next=head;

    head=node;

    return head;
}
```

Insert at End

Traverse till last

Attach node

```
ListNode* insertEnd(ListNode* head,int val){

    ListNode* node=new ListNode(val);

    if(head==NULL)
        return node;

    ListNode* temp=head;

    while(temp->next)
        temp=temp->next;

    temp->next=node;

    return head;
}
```

Delete First Node

```
ListNode* deleteFront(ListNode* head){

    if(head==NULL)
        return NULL;

    ListNode* temp=head;

    head=head->next;

    delete temp;

    return head;
}
```

Delete Last Node

```
ListNode* deleteEnd(ListNode* head){

    if(head==NULL)
        return NULL;

    if(head->next==NULL){
        delete head;
        return NULL;
    }

    ListNode* temp=head;

    while(temp->next->next)
        temp=temp->next;

    delete temp->next;

    temp->next=NULL;

    return head;
}
```

Most Important Traversal Template

```
ListNode* temp = head;

while(temp){

    // Process current node

    temp = temp->next;
}
```

Floyd's Cycle Detection Algorithm.

```

class Solution {
public:
    ListNode* reverseList(ListNode* head) {

        ListNode* prev = NULL;
        ListNode* curr = head;
        ListNode* next = NULL;

        while (curr) {

            next = curr->next;    // Save next node

            curr->next = prev;    // Reverse Link

            prev = curr;         // Move prev

            curr = next;         // Move curr

        }

        return prev;
    }
};

```

HEAP

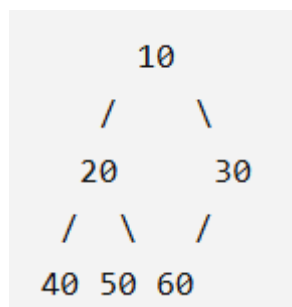
What is a Heap?

A Heap is a Complete Binary Tree that follows a special property.

Complete Binary Tree

Every level is completely filled except possibly the last level.

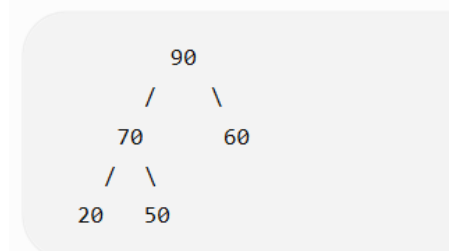
The last level is filled from left to right.



There are two kinds.

1. Max Heap

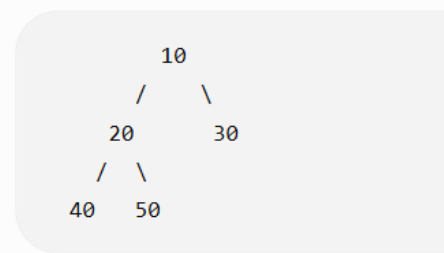
Parent is greater than children.



Every parent $\geq$ children.

2. Min Heap

Parent is smaller than children.



Every parent $\leq$ children.

Heap is **NOT sorted**.

Only parent-child relation matters.

PARENT FOR ITH IDX : $\text{parent} = (i-1)/2$

LEFT CHILD : $2*i+1$

RIGHT CHILD : $2*i+2$

STL

`priority_queue<int> pq;`

THIS IS MAX HEAP

```

priority_queue<int> pq;

pq.push(5);
pq.push(2);
pq.push(10);
pq.push(7);

cout << pq.top();

```

Output

10

MIN HEAP:

```

priority_queue<
int,
vector<int>,
greater<int>
> pq;

```

```

priority_queue<
int,
vector<int>,
greater<int>
> mn;

mn.push(3);
mn.push(8);
mn.push(1);

while(!mn.empty())
{
    cout<<mn.top()<<" ";
    mn.pop();
}

```

Output

```
1 3 8
```

Use a **heap** when you repeatedly need the **minimum or maximum** element while the data changes dynamically. If you only need the minimum/maximum once, a simple linear scan is usually enough.

HEAP SORT

```

class Solution {
public:

    void heapify(vector<int>& nums, int n, int i){

        int largest=i;

        int left=2*i+1;
        int right=2*i+2;

        if(left<n && nums[left]>nums[largest])
            largest=left;

        if(right<n && nums[right]>nums[largest])
            largest=right;

        if(largest!=i){
            swap(nums[i],nums[largest]);
            heapify(nums,n,largest);
        }
    }

    void heapSort(vector<int>& nums){

        int n=nums.size();

        // Build Max Heap
        for(int i=n/2-1;i>=0;i--){
            heapify(nums,n,i);
        }

        // Sorting
        for(int i=n-1;i>0;i--){

            swap(nums[0],nums[i]);

            heapify(nums,i,0);
        }
    }
};

```

GRAPH:

A Graph is a collection of vertices (nodes) connected by edges.

Terminology:

1] node : vertex point

2] edge- Connection between two vertices.

3] degree : Number of edges connected to a node.

4] path : A sequence of connected vertices

5] cycle : A path that starts and ends at the same node.

6] connected graph : Every node can be reached from every other node

7] disconnected graph : two different

8] connected component : Each separate connected part of a graph

Types of graph

1. Undirected Graph ★

No direction.

A ----- B

means

A ↔ B

If A can reach B,

then B can also reach A.

2. Directed Graph (Digraph)

Direction matters.

A -----> B

This does not mean

B -----> A

3. Weighted Graph

Edges contain values.

The numbers represent, Distance ,Cost ,Time ,Fuel ,Price depending on the problem.

4. Unweighted Graph

Every edge is considered to have equal cost.

Each edge has an implicit weight of 1.

Trees vs Graphs

Tree	Graph
No cycles	May have cycles
Connected	Can be disconnected
Exactly $n-1$ edges	Any number of edges
One path between two nodes	Multiple paths possible
One root	Usually no root
A tree is a special type of graph.	
Every tree is a graph.	
Every graph is not a tree.	

Representation Preview

1. Edge List

Simply store every edge.

```
</> C++  
  
vector<pair<int,int>> edges;  
  
edges.push_back({1,2});  
edges.push_back({1,3});  
edges.push_back({2,4});  
edges.push_back({3,4});  
edges.push_back({4,5});
```

Advantages

- Very easy
- Used in Kruskal's Algorithm

2. Adjacency Matrix

```
</> C++  
  
int n = 5;  
vector<vector<int>> matrix(n + 1, vector<int>(n + 1, 0));  
  
matrix[1][2] = matrix[2][1] = 1;  
matrix[1][3] = matrix[3][1] = 1;  
matrix[2][4] = matrix[4][2] = 1;  
matrix[3][4] = matrix[4][3] = 1;  
matrix[4][5] = matrix[5][4] = 1;
```

Advantages

Checking whether an edge exists:

Very fast $O(1)$

Disadvantages

Space:

$O(V^2)$

3. Adjacency List

This is the representation used in 95% of interview and competitive programming problems.

Think of each node maintaining a list of its neighbors.

```
int n, m;  
cin >> n >> m;  
  
vector<vector<int>> adj(n + 1);  
  
for(int i = 0; i < m; i++)  
{  
    int u, v;  
    cin >> u >> v;  
  
    adj[u].push_back(v);  
    adj[v].push_back(u);  
}
```

Add edge 1-2

```
</> C++  
  
adj[1].push_back(2);  
adj[2].push_back(1);
```

Directed Graph Representation

Suppose the graph is:

```
1 → 2  
1 → 3  
2 → 4
```

Only store the directed edge:

```
</> C++  
  
adj[1].push_back(2);  
adj[1].push_back(3);  
adj[2].push_back(4);
```

Weighted Graph Representation

Suppose:

```
1 --5-- 2  
1 --2-- 3
```

Store both the neighbor and its weight:

```
</> C++  
  
vector<vector<pair<int,int>>> adj(n + 1);  
  
adj[1].push_back({2,5});  
adj[2].push_back({1,5});  
  
adj[1].push_back({3,2});  
adj[3].push_back({1,2});
```

Traversal:

```
</> C++  
  
for(auto [neighbor, weight] : adj[node])  
{  
    cout << neighbor << " " << weight << endl;  
}
```

Complexity Comparison

Representation	Space	Check Edge	Traverse Neighbors
Edge List	$O(E)$	$O(E)$	$O(E)$
Adjacency Matrix	$O(V^2)$	$O(1)$	$O(V)$
Adjacency List	$O(V + E)$	$O(\text{degree})$	$O(\text{degree})$ ★

For almost every graph problem in interviews and competitive programming, **Adjacency List** is the preferred representation.

BFS

Mark source visited

Push source into queue

While queue is not empty

Remove front node

Process node

For every neighbor

If not visited

Mark visited

Push into queue

BFS always explores all nodes at distance 1, then distance 2, then distance 3, and so on.

That's why BFS is the algorithm used to find the shortest path in an unweighted graph.

BFS can only travel to nodes connected to the starting node.

It **cannot jump** between disconnected components.

Run BFS from **every unvisited node**.

Instead of

```
</> C++
```

```
bfs(1);
```

we do

```
</> C++
```

```
for(int i = 1; i <= n; i++)
{
    if(!visited[i])
    {
        bfs(i);
    }
}
```

Generic BFS for Entire Graph

```
</> C++
```

```
void bfs(int start, vector<vector<int>>& adj, vector<bool>& visited)
{
    queue<int> q;

    q.push(start);
    visited[start] = true;

    while(!q.empty())
    {
        int node = q.front();
        q.pop();

        cout << node << " ";

        for(int neighbor : adj[node])
        {
            if(!visited[neighbor])
            {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
    }
}
```

Main function:

```
</> C++
```

```
int n, m;
cin >> n >> m;

vector<vector<int>> adj(n + 1);

for(int i = 0; i < m; i++)
{
    int u, v;
    cin >> u >> v;

    adj[u].push_back(v);
    adj[v].push_back(u);
}

vector<bool> visited(n + 1, false);

for(int i = 1; i <= n; i++)
{
    if(!visited[i])
    {
        bfs(i, adj, visited);
    }
}
```

DFS

Mark current node visited

Process current node

For every neighbor

If not visited

DFS(neighbor)

```
void dfs(int node,
        vector<vector<int>>& adj,
        vector<bool>& visited)
{
    visited[node] = true;

    cout << node << " ";

    for(int neighbor : adj[node])
    {
        if(!visited[neighbor])
        {
            dfs(neighbor, adj, visited);
        }
    }
}
```

```
return 1
    + dfs(up)
    + dfs(down)
    + dfs(left)
    + dfs(right);
```

CYCLE DETECTION

Cycle Detection in Undirected Graph

Cycle means Starting from a node, you can return to the same node **without using the same edge twice**.

Parent Concept

Every node remembers who discovered it.

Rule ★★★★★

During BFS:

If

```
<< C++
visited[neighbor]
```

AND

```
<< C++
neighbor != parent
```

Then

```
Cycle Found
```

This single condition is the heart of undirected cycle detection.

Pseudocode

```
Push(start,-1)

visited[start]=true

While queue not empty

    node,parent = front

    For every neighbor

        If not visited

            visited

            Push(neighbor,node)

        Else if(neighbor != parent)

            return true

return true
```

```
bool bfs(int start, vector<vector<int>>& adj, vector<bool>& visited)
{
    queue<pair<int, int>> q;

    q.push({start, -1});
    visited[start] = true;

    while (!q.empty())
    {
        auto [node, parent] = q.front();
        q.pop();

        for (int neighbor : adj[node])
        {
            if (!visited[neighbor])
            {
                visited[neighbor] = true;
                q.push({neighbor, node});
            }
            else if (neighbor != parent)
            {
                return true;
            }
        }
    }

    return false;
}
```

Whole Graph (Disconnected Graph)

```
<< C++

bool isCycle(int V, vector<vector<int>>& adj)
{
    vector<bool> visited(V, false);

    for (int i = 0; i < V; i++)
    {
        if (!visited[i])
        {
            if (bfs(i, adj, visited))
                return true;
        }
    }

    return false;
}
```

DFS Cycle Detection (Undirected Graph)

Rule ★★★★★

Exactly the same as BFS.

```
<> C++  
  
if (!visited[neighbor])  
{  
    dfs(neighbor, node);  
}  
else if (neighbor != parent)  
{  
    return true;  
}
```

DFS Code

```
<> C++  
  
bool dfs(int node,  
        int parent,  
        vector<vector<int>>& adj,  
        vector<bool>& visited)  
{  
    visited[node] = true;  
  
    for (int neighbor : adj[node])  
    {  
        if (!visited[neighbor])  
        {  
            if (dfs(neighbor, node, adj, visited))  
                return true;  
        }  
        else if (neighbor != parent)  
        {  
            return true;  
        }  
    }  
  
    return false;  
}
```

```
bool isCycle(int V, vector<vector<int>>& adj)  
{  
    vector<bool> visited(V, false);  
  
    for (int i = 0; i < V; i++)  
    {  
        if (!visited[i])  
        {  
            if (dfs(i, -1, adj, visited))  
                return true;  
        }  
    }  
  
    return false;  
}
```

BFS	DFS
Queue	Recursion
(node,parent) stored in queue	parent passed as parameter

TOPOLOGICAL SORT

Lesson 1: What is Topological Sort?

Topological Sort is an ordering of vertices such that:

For every directed edge $u \rightarrow v$, u appears before v in the ordering.

Notice the keyword:

★ Only for Directed Acyclic Graphs (DAG)

Not for:

- ✗ Undirected Graphs
- ✗ Directed graphs with cycles

Topological sort is **not unique**.

As long as every edge goes from left to right, the order is valid.

A graph containing a cycle has no topological ordering.

DFS Idea ★★★★★

This is the key insight.

Normally DFS does:

```
1  
↓  
2  
↓  
3
```

When DFS finishes exploring 3, it returns.

Then 2 returns.

Then 1 returns.

The finishing order is:

```
3  
↓  
2  
↓  
1
```

If we save nodes **when DFS finishes**, we get:

```
3 2 1
```

Reverse it:

```
1 2 3
```

That's the Topological Order.

Why Push After DFS?

Example:

```
1 → 2 → 3
```

If we push **before** DFS:

```
1 2 3
```

This works here, but fails for more complex graphs.

Instead, we push **after** visiting all children.

Example:

```
// C++
dfs(1)
  dfs(2)
    dfs(3)

    push(3)

  push(2)

push(1)
```

DFS Algorithm

1. Mark node visited.
2. Visit all neighbors.
3. Push current node into a stack.

After all DFS calls finish:

Pop the stack.

That is the topological order.

```
void dfs(int node,
        vector<vector<int>>& adj,
        vector<bool>& visited,
        stack<int>& st)
{
    visited[node] = true;

    for(int neighbor : adj[node])
    {
        if(!visited[neighbor])
        {
            dfs(neighbor, adj, visited, st);
        }
    }

    // Push AFTER exploring children
    st.push(node);
}
```

```
vector<int> topoSort(int V, vector<vector<int>>& adj)
{
    vector<bool> visited(V, false);
    stack<int> st;

    for(int i = 0; i < V; i++)
    {
        if(!visited[i])
            dfs(i, adj, visited, st);
    }

    vector<int> ans;

    while(!st.empty())
    {
        ans.push_back(st.top());
        st.pop();
    }

    return ans;
}
```

Unlike normal DFS:

```
// C++

visited[node] = true;

for(neighbor)

    dfs(neighbor);
```

Topological DFS adds **one extra line**:

```
// C++

st.push(node);
```

after exploring all neighbors.

That one line is the heart of the algorithm.

Kahn's Algorithm (BFS Topological Sort)

What is Indegree?

Indegree of a node = Number of incoming edges.

Example:

```
0 → 1 ← 2
```

Indegree:

```
0 = 0
```

```
1 = 2
```

```
2 = 0
```

Algorithm

Step 1

Calculate indegree of every node.

Step 2

Push every node with indegree 0 into a queue.

Step 3

Pop a node.

Add it to the answer.

Step 4

For every neighbor:

```
⟨⟩ C++
```

```
indegree[neighbor]--;
```

If indegree becomes 0:

```
⟨⟩ C++
```

```
push into queue
```

Repeat until the queue is empty.

Computing Indegree

Initially:

```
⟨⟩ C++
```

```
vector<int> indegree(V, 0);
```

Traverse adjacency list:

```
⟨⟩ C++
```

```
for(int i = 0; i < V; i++)
{
    for(int neighbor : adj[i])
    {
        indegree[neighbor]++;
    }
}
```

Push Initial Nodes

```
⟨⟩ C++
```

```
queue<int> q;

for(int i = 0; i < V; i++)
{
    if(indegree[i] == 0)
    {
        q.push(i);
    }
}
```

```
vector<int> topoSort(int V, vector<vector<int>>& adj)
{
    vector<int> indegree(V, 0);

    for(int i = 0; i < V; i++)
    {
        for(int neighbor : adj[i])
        {
            indegree[neighbor]++;
        }
    }

    queue<int> q;

    for(int i = 0; i < V; i++)
    {
        if(indegree[i] == 0)
            q.push(i);
    }

    vector<int> ans;

    while(!q.empty())
    {
        int node = q.front();
        q.pop();

        ans.push_back(node);

        for(int neighbor : adj[node])
        {
            indegree[neighbor]--;

            if(indegree[neighbor] == 0)
            {
                q.push(neighbor);
            }
        }
    }

    return ans;
}
```



Why Is This Better?

Kahn's Algorithm can detect a cycle **without any extra logic**

Suppose:

```
0 → 1 → 2
↑       |
└──────┘
```

Indegree:

0 = 1

1 = 1

2 = 1

There is **no node with indegree 0**.

Queue starts empty.

No node is processed.

This immediately indicates a cycle.

PATH

```
vector<int> shortestPath(int V,
                        vector<vector<int>>& adj,
                        int src)
{
    vector<int> dist(V, -1);

    queue<int> q;

    q.push(src);
    dist[src] = 0;

    while (!q.empty())
    {
        int node = q.front();
        q.pop();

        for (int neighbor : adj[node])
        {
            if (dist[neighbor] == -1)
            {
                dist[neighbor] = dist[node] + 1;
                q.push(neighbor);
            }
        }
    }

    return dist;
}
```

Graph Type	Algorithm	Time
Unweighted Graph	BFS	$O(V+E)$
Weighted DAG	Topological Sort + Relaxation	$O(V+E)$
General Weighted Graph (non-negative weights)	Dijkstra	$O(E \log V)$

1 Shortest Path in Unweighted Graph (BFS)

Shortest Path in Unweighted Graph (BFS) - Algorithm

1. Create a distance array and initialize all values to -1.
2. Create a queue.
3. Push the source node into the queue.
4. Set $\text{distance}[\text{source}] = 0$.
5. While the queue is not empty:
 - a) Pop the front node.
 - b) Traverse all its neighbors.
 - c) If the neighbor has not been visited ($\text{distance}[\text{neighbor}] == -1$):
 - $\text{distance}[\text{neighbor}] = \text{distance}[\text{node}] + 1$
 - Push the neighbor into the queue.
6. After BFS finishes, the distance array contains the shortest distance from the source to every node.
7. Return the distance array.

```
dist[] = -1

queue

push(source)

dist[source]=0

while(queue not empty)

    node = pop()

    for(neighbour)

        if(dist[neighbour]==-1)

            dist[neighbour]=dist[node]+1

            push(neighbour)
```

2 Shortest Path in Weighted DAG

1. Topologically sort the graph.
2. Initialize all distances to INF.
3. $\text{dist}[\text{src}] = 0$.
4. Traverse vertices in topological order.
5. For every outgoing edge:
 - relax it.
6. Return distance array.

Relaxation = "Try to improve the shortest distance."

Dijkstra's Algorithm

Always process the currently nearest unprocessed node.

This is the greedy idea behind Dijkstra.

1. Initialize `dist[] = INF`.
2. `dist[source] = 0`.
3. Push `(0, source)` into a Min Heap.
4. While the heap is not empty:
 - a. Pop the node with the minimum distance.
 - b. Traverse all its neighbors.
 - c. Relax each edge.
 - d. If a distance improves, push the updated `(distance, node)` into the heap.
5. Return `dist[]`.

The only difference from DAG is how we choose the next node:

- DAG: Topological order
- Dijkstra: Min Heap (smallest current distance)
- Because **all edge weights are non-negative**.
- When the node with the smallest distance is removed from the min heap, **no future path can make its distance smaller**. Any alternative path would have to pass through another node whose distance is already greater than or equal to the current one, and adding a non-negative edge cannot reduce the total cost.

1. Create a distance array and initialize all distances to INF.
2. Set `dist[source] = 0`.
3. Create a Min Heap storing `(distance, node)`.
4. Push `(0, source)` into the Min Heap.
5. While the Min Heap is not empty:
 - a. Pop the node with the smallest distance.
 - b. If this distance is outdated, skip it.
 - c. Traverse all adjacent nodes.
 - d. Relax every edge.
 - e. If distance improves, update `dist[]` and push the new pair.
6. Return `dist[]`.

```
class Solution {
public:
    vector<int> dijkstra(int V, vector<vector<int>>> adj[], int S) {

        priority_queue<pair<int,int>,
                      vector<pair<int,int>>,
                      greater<pair<int,int>>> pq;

        vector<int> dist(V, 1e9);

        dist[S] = 0;
        pq.push({0, S});

        while (!pq.empty()) {

            int dis = pq.top().first;
            int node = pq.top().second;
            pq.pop();

            // Ignore outdated entry
            if (dis > dist[node])
                continue;

            for (auto it : adj[node]) {

                int adjNode = it[0];
                int wt = it[1];

                if (dist[node] + wt < dist[adjNode]) {

                    dist[adjNode] = dist[node] + wt;
                    pq.push({dist[adjNode], adjNode});
                }
            }
        }

        return dist;
    }
};
```